Evaluating AI Agents with LLM-Based Judges

Prashant Kafle
Department of Computer Engineering
Thapathali Campus, IOE
Kathmandu, Nepal
Roll no: THA080BCT026
prashantkafle7738@gmail.com

Sunil Paudel
Department of Computer Engineering
Thapathali Campus, IOE
Kathmandu, Nepal
Roll no: THA080BCT047
sunil.080bct047@tcioe.edu.np

***Abstract*—As language-model agents take on longer tasks, evaluating their outputs becomes a practical systems problem. Human assessment is expensive and difficult to repeat at scale, while automatic metrics can miss factual errors, useful reasoning, and task-specific quality. This paper reports on two linked but methodologically separate studies conducted on a shared benchmark of agent tasks sampled from GAIA [13]. Study 1 is a purely algorithmic comparison: ROUGE-L measures reference-based similarity with longest common subsequence dynamic programming, Bradley–Terry and Elo-style models convert pairwise judgments into rankings, and SelfCheckGPT uses disagreement among stochastic samples to detect possible hallucinations, applied to 97 agent responses generated from 97 stratified GAIA tasks by a single agent configuration, together with a $k = 3$ resampling of an 8-task subset used for the SelfCheckGPT and per-task Bradley–Terry analysis, and a supplementary 50-task cross-backend ROUGE-L comparison against a second agent configuration built on Google's Gemini. Study 2 is a human–LLM judge agreement study: two trained human annotators and an LLM judge independently label a 30-item pool drawn from those responses and resamples under a fixed four-category rubric, and Cohen's kappa measures whether the judge's decisions agree with a human-adjudicated standard beyond chance. The two studies share a single research question – whether an LLM judge can be treated as a reliable measurement instrument rather than merely a convenient source of scores – but are designed, sampled, and analyzed independently, so that an algorithmic result from Study 1 is never mistaken for evidence of judge validity in Study 2.**

***Index Terms*—agent evaluation, LLM-as-a-judge, ROUGE-L, Bradley–Terry model, SelfCheckGPT, Cohen's kappa**

I. Introduction

Modern agents can plan, call tools, and revise their own outputs. That makes evaluation harder than checking whether a short answer matches a reference. An agent may produce a correct final answer with different wording, or a plausible answer that contains one unsupported claim. It may also succeed on one run and fail on another because sampling, tool responses, or intermediate decisions change. A useful evaluation method therefore needs to measure more than surface similarity.

Human review remains the most direct way to judge whether an agent followed instructions and produced a useful result. It is slow, expensive, and difficult to apply consistently across many runs. This has encouraged the use of automatic metrics and, more recently, large language models as evaluators. An LLM judge can read an open-ended response and return a score, a comparison, or a written explanation. The same flexibility that makes this approach attractive also creates a problem: the judge may favor certain styles, follow superficial cues, or agree with another model without tracking factual correctness.

This paper treats evaluation as a sequence of related algorithmic problems. First, ROUGE-L provides a low-cost reference-based score using the longest common subsequence (LCS). Second, pairwise model comparisons can be aggregated with Bradley–Terry or Elo-style ranking models. Third, SelfCheckGPT tests whether sampled answers remain consistent when the system has no external fact database. Finally, Cohen's kappa measures agreement between the automated judge and human annotations. The last step is essential. A judge that produces stable scores but disagrees with people is not necessarily reliable.

The study will implement these methods and compare their computational behavior and practical limitations on agent-generated responses. The main research question is: *under what conditions do automatic and LLM-based evaluation methods provide measurements that are both scalable and aligned with human judgment?* The planned analysis has four goals:

- implement ROUGE-L and report its dynamic-programming cost and sensitivity to wording;
- estimate model rankings from pairwise judgments and examine how rankings change with the comparison data;
- test whether SelfCheckGPT's consistency signal identifies hallucinated or unsupported claims; and
- measure judge–human agreement with Cohen's kappa instead of treating an LLM score as ground truth.

The remainder of the paper reviews prior work, describes the algorithms and experimental design, reports results, and discusses the limits of automated evaluation.

II. Literature Review

A. Two Layers of Evaluation

Prior work on evaluating language-model agents can be separated into two layers that are frequently discussed together but ask different questions. The first layer evaluates *agent outputs*: does a response match a reference, does it rank above a competing response, and is it internally consistent enough

to be trusted as a factual claim? The second layer evaluates *the evaluator itself*: when an LLM is used to produce those judgments, is the judge's own output reliable, and does it agree with independent human decisions? Blending these two layers makes it easy to mistake a stable or fluent judgment for a correct one. This paper's four-part pipeline follows the same split: ROUGE-L, the Bradley–Terry/Elo ranking models, and SelfCheckGPT belong to the first layer, while LLM-as-a-judge reliability and Cohen's kappa belong to the second. The rest of this section reviews prior work under each layer in turn.

B. Layer 1: Evaluating Agent Outputs

1) Reference-Based Text Evaluation: ROUGE was introduced as a package for comparing generated summaries with human-written references [1]. ROUGE-L uses the longest common subsequence between a candidate and a reference. Unlike exact n-gram overlap, the LCS can reward a shared sequence even when words are separated by insertions. Its dynamic-programming computation is straightforward: for sequences of lengths m and n, the standard LCS table requires $O(mn)$ time and $O(mn)$ space, or $O(\min(m, n))$ space with row reuse. These properties make ROUGE-L useful as a reproducible baseline.

Its limitation is equally important for agent evaluation. ROUGE-L measures overlap with a reference, not whether a response is factually correct, executable, or useful. Two valid answers can receive different scores because they use different phrasing. A response can also share many words with a reference while containing a subtle error. Recent surveys therefore place lexical metrics among the efficient tools for constrained tasks, while noting their weakness on open-ended generation [6].

2) Pairwise Comparison and Ranking: When a single numeric reference score is unavailable, evaluators can compare two outputs directly. The Bradley–Terry model represents the probability that item i defeats item j as

$$P(i \succ j) = \frac{\exp(\beta_i)}{\exp(\beta_i) + \exp(\beta_j)}, \tag{1}$$

where β_i and β_j are latent quality parameters [2]. Fitting these parameters over many comparisons produces a ranking and exposes uncertainty caused by sparse or uneven comparisons. Elo ratings use a related update rule and are widely associated with competitive pairwise ranking [3].

Pairwise evaluation is attractive because people often find it easier to choose the better of two answers than to assign an absolute score. This idea appears in large-scale language-model evaluation, including MT-Bench and Chatbot Arena, where model comparisons are aggregated into rankings [4]. However, rankings inherit the judge's biases. Position effects, verbosity preferences, and inconsistent criteria can change the result. A ranking algorithm can summarize judgments accurately while the judgments themselves remain poorly grounded – a concern that belongs to the second layer of evaluation discussed below.

3) Consistency-Based Hallucination Detection: SelfCheckGPT addresses hallucination detection without requiring model probabilities or an external knowledge base [5]. The method samples multiple responses to the same prompt and compares the claims across samples. If a model has stable knowledge of a fact, the samples should tend to agree. Unsupported claims are more likely to vary or contradict one another. The approach can therefore be applied to black-box systems for which token probabilities are unavailable.

The method does not establish truth by itself. A model can repeat the same false statement in every sample, and legitimate answers may vary in wording. Sampling also adds computation: producing k responses costs roughly k model calls before the consistency comparisons are performed. These tradeoffs make SelfCheckGPT a useful diagnostic signal rather than a replacement for fact verification. The proposed study will examine both its detection quality and its cost as the number of samples changes.

C. Layer 2: Evaluating the Evaluators

1) LLM-as-a-Judge and Its Own Reliability: LLM-as-a-judge systems use a language model to score, rank, or select outputs, and increasingly stand in for the ranking and consistency judgments described in Layer 1. Surveys of the area describe applications in open-ended generation and identify reliability, bias, prompt design, and judge calibration as unresolved concerns [7], [8]. The central distinction is between using an LLM to generate a judgment and showing that the judgment is valid. A fluent explanation or a stable score does not prove agreement with a human evaluator, which is precisely why the judge itself, and not only the outputs it scores, must be treated as an object of study.

2) Benchmarks That Test Judges Directly: A growing line of work targets the judge as the unit of evaluation rather than the agent under test. JudgeBench constructs challenging response pairs with objectively correct preference labels and shows that strong LLM judges, including GPT-4o, perform only slightly better than random guessing on cases where crowdsourced preference is a poor proxy for factual and logical correctness [10]. Agent-as-a-Judge extends this idea to agentic, multi-step tasks: it uses an agentic evaluator that inspects intermediate steps rather than only the final output, and reports that this step-aware judge is more reliable than a conventional LLM-as-a-judge baseline and comparable to human evaluation on the DevAI benchmark [11]. AgentRewardBench evaluates LLM judges specifically on web-agent trajectories against expert annotations and finds that no single judge performs well across all benchmarks, and that the rule-based evaluators commonly used as a substitute tend to underreport agent success [12]. Together, these benchmarks make the same point from three different task settings: a judge's fluency or apparent consistency is not evidence of its correctness, and judges require dedicated benchmarks in the same way that agents do.

3) Chance-Corrected Human Agreement: Cohen's kappa provides a lightweight way to test judge–human agreement for

categorical labels while correcting for agreement expected by chance [9]. For observed agreement p_o and chance agreement p_e, it is defined as

$$\kappa = \frac{p_o - p_e}{1 - p_e}. \tag{2}$$

Unlike raw accuracy, kappa can reveal when a judge appears accurate mainly because one label dominates the dataset. Its interpretation still depends on the label distribution and annotation protocol, so it should be reported with the confusion matrix and class counts. Compared with dedicated judge benchmarks such as JudgeBench and AgentRewardBench, kappa is a much smaller-scale check, but it plays the same role: it is the validation measure that connects the Layer 1 algorithms to the practical question of trust, namely whether an automated judge agrees with human decisions on the same outputs.

D. Research Gap

Existing work tends to stay within one of the two layers, and this separation is the concrete gap this paper targets. Reference metrics, pairwise ranking models, and hallucination detectors study the outputs of an agent; judge benchmarks such as JudgeBench, Agent-as-a-Judge, and AgentRewardBench study the reliability of the judge in isolation, typically on a single task family such as knowledge-and-reasoning pairs, coding, or web-agent trajectories [10]–[12]. None of these studies test whether the same LLM judge that produces Layer 1 pairwise labels (feeding the Bradley–Terry ranking) or Layer 1 consistency judgments is also validated, on the same task set, against independent human labels. This is a specific and currently unanswered empirical question, not a rhetorical one: a judge could produce Bradley–Terry rankings that remain stable across bootstrap resamples of the comparison data while still disagreeing with human annotators beyond what chance predicts, and no published study we found reports both quantities on the same benchmark. The four methods studied here are therefore not grouped by convenience; they are grouped because Study 2 (Section IV-D) directly tests the judge that Study 1 (Section IV-C) partially depends on to produce its pairwise and consistency labels. The two studies are reported separately, on the same GAIA-derived benchmark, precisely so that a favorable result in one is never substituted for evidence in the other.

III. Theoretical Framework and Justification

The four methods studied in this paper are not arbitrary choices. Each targets a distinct layer of the evaluation problem, and together they form a chain that moves from surface-level text comparison to a validated measure of judge trustworthiness. This section states the theoretical basis for that chain and justifies why each algorithm is necessary rather than redundant.

A. Evaluation as a Measurement Problem

Treating agent evaluation as a measurement problem, rather than a scoring exercise, motivates the overall design. A measurement instrument must satisfy two properties: it must be computable at reasonable cost, and its output must correspond to the quantity it claims to measure. Classical test theory separates these concerns into reliability, whether repeated measurement gives consistent results, and validity, whether the measurement reflects the target construct [9]. ROUGE-L and Bradley–Terry/Elo scores are efficient and reliable by construction, since they are deterministic functions of fixed inputs. Their validity, that is, whether a high score reflects real response quality, is not guaranteed and must be checked separately. SelfCheckGPT targets a different construct, factual consistency, that similarity and ranking scores do not capture. Cohen's kappa is included specifically to test validity: it measures whether the judge's categorical decisions align with human decisions beyond chance, which reliability alone cannot establish. This framework explains why no single metric in the pipeline is sufficient on its own.

B. Complexity and Feasibility Justification

Each algorithm is chosen partly because its computational profile matches what is feasible to run repeatedly over an agent's outputs. The LCS dynamic program behind ROUGE-L runs in $O(mn)$ time, which is negligible next to the cost of generating the responses themselves, so it is used as a cheap first-pass filter. Bradley–Terry and Elo estimation are used because pairwise comparison data can be collected incrementally, and maximum-likelihood fitting of the Bradley–Terry model or iterative Elo updates scale close to linearly in the number of comparisons, which keeps ranking updates tractable as more agent outputs are produced. SelfCheckGPT is more expensive, since it requires k additional model samples per prompt, so it is justified only for the subset of outputs where a similarity or ranking score is inconclusive, not for every response. Cohen's kappa is computed once over a fixed confusion matrix and adds negligible cost, which is why it is used as the final validation step rather than an early filter. Ordering the pipeline from cheapest to most expensive is itself a design decision grounded in algorithmic complexity, not only in what each method measures.

C. Why Pairwise Comparison Over Absolute Scoring

The choice of Bradley–Terry and Elo models over direct absolute scoring follows from known limitations of asking a judge, human or LLM, to assign a number on a fixed scale. Absolute scores from different judges or different sessions are not guaranteed to be on a common scale, and a single judge's scale can drift across a long batch of items. Pairwise comparison avoids anchoring an output to an arbitrary numeric scale; it only requires a local decision about which of two outputs is better, and the ranking is recovered afterward through the probabilistic model in Equation (1). This is the same justification used in large-scale human preference benchmarking [4], and it applies equally when the judge is an LLM rather than a person.

D. Why Consistency Substitutes for Ground Truth

SelfCheckGPT is justified by a specific theoretical assumption: when a language model has genuinely learned a fact,

resampling under different random seeds should reproduce that fact, while an unsupported or fabricated claim is less likely to be reproduced identically across samples [5]. This assumption lets the method detect a likely hallucination without an external database, at the cost of being a proxy rather than a direct correctness check. The framework used here treats SelfCheckGPT's output as an additional feature alongside ROUGE-L and ranking scores, not as a standalone verdict, which is consistent with its original design as a sampling-based diagnostic rather than a certifier of truth.

E. Why Human Agreement Is the Final Criterion

The theoretical framework treats the LLM judge as a hypothesis to be tested, not as an oracle. This follows directly from inter-rater reliability theory, where any single rater's output is only meaningful once compared against another rater using a chance-corrected statistic [9]. Applying Cohen's kappa between the LLM judge and human annotators operationalizes this idea: a kappa near zero indicates that apparent judge accuracy is an artifact of label imbalance, while a high kappa supports treating the judge's categorical decisions as a usable substitute for costly human review in future evaluations. This is why the pipeline ends with kappa rather than with the judge's own score.

F. Why Two Studies Rather Than One Pipeline

The four methods belong together theoretically, but they do not belong in a single empirical design. ROUGE-L, Bradley–Terry/Elo, and SelfCheckGPT are deterministic or sampling-based algorithms applied to fixed inputs; evaluating them requires no human subjects and answers questions about computational cost and statistical behavior. Cohen's kappa evaluates a different kind of claim, namely whether a specific judge's categorical decisions agree with independent human decisions, and answering it requires a human-subjects annotation protocol with its own sampling, blinding, and reliability checks. Reporting these under one undifferentiated "Research Design" section, as an earlier draft of this paper did, made the four methods read as unrelated case studies rather than as a chain. Section IV-C therefore reports Study 1 (the algorithmic comparison) on its own data subsample and its own statistical tests, and Section IV-D reports Study 2 (the judge-agreement study) on its own subsample, annotation protocol, and statistical tests. The two studies are connected only through the research question stated in Section I and through the fact that the LLM judge validated in Study 2 is the same judge used to generate the pairwise labels analyzed in Study 1.

IV. Research Design

A. Overview

The study is organized as two separate studies that share a single source of agent tasks (Section IV-B) but differ in design, sampling, and analysis. Study 1 (Section IV-C) is a purely algorithmic comparison of ROUGE-L, Bradley–Terry/Elo ranking, and SelfCheckGPT; it requires no human subjects and answers questions about computational cost and statistical behavior. Study 2 (Section IV-D) is a human–LLM judge agreement study; it requires an independent human-annotation protocol and answers whether the LLM judge agrees with human decisions beyond chance. Results, procedures, and statistical tests for the two studies are reported in separate subsections so that they are never conflated.

B. Shared Task Source and Inclusion Criteria

Both studies draw their agent tasks from the validation split of the GAIA benchmark [13], which provides 165 questions with public, human-verified reference answers across three difficulty levels (Level 1: single-step lookup; Level 2: short multi-step reasoning or tool use; Level 3: long multi-step tool chains). GAIA is used instead of a self-constructed task set because its reference answers and difficulty stratification are already documented and independently verifiable, which supports the reproducibility of task selection.

A GAIA task is included in the study if it satisfies all of the following criteria:

- it is text-only, i.e. answerable using only the question text and web search, with no attached image, audio, or document file (the evaluation sandbox provides a web-search tool, not general file processing);
- its published reference answer is a short, checkable string suitable for exact-match or ROUGE-L scoring;
- the agent under evaluation returns a non-empty final answer using a bounded budget of web-search calls and a single LLM call.

Applying these criteria to the GAIA validation split yielded a working set of 97 tasks, stratified across the three difficulty levels (35 Level 1, 45 Level 2, and 17 Level 3 tasks). Each included task is run through a single agent configuration – one LLM call (Groq-hosted Llama 3.3 70B) augmented with DuckDuckGo web-search snippets – producing 97 agent responses that form the base dataset for Study 1's reference-based scoring. The original proposal compared two agent configurations (a baseline and a tool-augmented variant); the study evaluates one primary agent configuration as its main analysis, supplemented by a secondary cross-backend comparison on a stratified 50-task subset of the same corpus, reported in Section IV-C, so that Study 1's low-score findings can be checked for backend-specificity rather than treated as a claim of generality across agent configurations. A further subset of 8 tasks, drawn from the 97-task corpus, is resampled $k = 3$ times to support the SelfCheckGPT consistency check and a per-task Bradley–Terry ranking of the three resamples. Study 2 (Section IV-D) draws its human-annotated pool from an earlier, fixed 18-task pilot collected before this corpus was expanded to 97 tasks for Study 1's automatic metrics; Study 2's sampling and results (Section V-B) are therefore unaffected by the later expansion and are reported on their own terms.

C. Study 1: Algorithmic Comparison of Output-Evaluation Methods

1) Design: Study 1 answers a purely algorithmic question: how do ROUGE-L, Bradley–Terry/Elo ranking, and Self-

CheckGPT behave, in terms of computational cost, score distribution, and ranking stability, on the same set of agent outputs? Study 1 uses no human labels and computes no Cohen's kappa; the pairwise labels it produces with the LLM judge are simply the object that Study 2 later validates.

2) Procedure:

- **Stage 1 – Reference scoring.** ROUGE-L is computed between each of the 97 agent responses and its GAIA reference answer using the LCS-based dynamic program. The full score distribution (mean, median, count of non-zero scores) is recorded.
- **Stage 1b – Backend comparison.** The same ROUGE-L procedure is repeated for a stratified 50-task subset (17 Level 1, 17 Level 2, 16 Level 3) of the 97-task corpus, run through a second agent configuration that substitutes Google's Gemini free-tier backend (`gemini-flash-latest`) for Groq's Llama 3.3 70B, with Groq's scores on the identical 50 tasks used as the paired baseline.
- **Stage 2 – Pairwise ranking.** Because only one agent configuration is evaluated (Section IV-B), Bradley–Terry ranking is applied per task instead of across configurations: for each task in the 8-task SelfCheckGPT subset, the LLM judge compares the $k = 3$ resampled responses pairwise, and a within-task Bradley–Terry strength is fit from the comparisons that survive a position-bias control (a comparison is kept only if the judge's verdict is unchanged when the response order is swapped).
- **Stage 3 – Consistency checking.** For the same 8-task, $k = 3$ resampled subset, SelfCheckGPT compares claims across the three samples per response and produces a consistency score per response.

3) Statistical Analysis Plan: ROUGE-L scores are summarized descriptively (mean, median, count of non-zero scores) over the 97 tasks, and separately over the 50-task Gemini-vs-Groq paired subset for the backend comparison. Because the primary Study 1 analysis evaluates one agent configuration at a time, the 50-task backend comparison is reported descriptively (paired means, medians, and non-zero counts on the identical task set) rather than as a formal hypothesis test, since a single non-bootstrapped comparison at $n = 50$ does not support one; the originally planned 500-resample bootstrap Kendall's τ over cross-configuration rankings is dropped for the same reason and reported here as a scope reduction rather than a result that was not run. Per-task Bradley–Terry fits are instead reported as a yield statistic: the fraction of the 8-task subset for which at least one position-bias-consistent comparison was obtained. SelfCheckGPT consistency scores are summarized descriptively and, for the same 8-task subset, compared qualitatively against the corresponding human hallucination labels from Study 2 as a small, non-circular sanity check; at $n = 8$ the subset is too small to support a formal correlation test.

D. Study 2: Human–LLM Judge Agreement Study

1) Design: Study 2 answers a different question from Study 1: does the LLM judge's categorical decision on a response agree with an independent human decision on the same response, beyond the agreement expected by chance? Study 2 is reported with its own subsample, its own annotators, and its own statistical test (Cohen's kappa), so that a favorable algorithmic result in Study 1 is never substituted for evidence of judge validity.

2) Sampling: A pool of 42 candidate items is built by combining the 18-task pilot responses with the 24 SelfCheckGPT resamples (8 tasks $\times$ $k = 3$) from Section IV-C, since the 18 main responses alone are fewer than the roadmap's 30-item annotation target. From this pool, 6 items are set aside for calibration and the remaining 30 are labeled by both annotators, each in an independently randomized order. This size keeps annotation load feasible for two annotators given the smaller, single-configuration task set actually used.

3) Rubric: Each response is labeled with exactly one of four categories, applied identically by both human annotators and the LLM judge: (1) *Correct* – matches the reference or task intent with no unsupported claims; (2) *Partially correct* – the main answer is right but includes a minor unsupported or extraneous claim; (3) *Incorrect* – the main answer does not match the reference or fails the task; (4) *Hallucinated* – the response contains a specific factual claim not supported by the reference, the retrieved tool output, or the task context.

4) Annotation Protocol: The two human annotators are the paper's authors. Because their shared involvement in the project could inflate agreement, the protocol includes the following controls:

- **Calibration.** Both annotators jointly label a separate calibration set of 6 responses (disjoint from the 30-item sample), discuss disagreements, and refine the rubric wording before the main annotation round.
- **Independent, blind labeling.** For the 30-item sample, each annotator labels all responses independently, in an independently randomized order, without access to the LLM judge's labels or to the other annotator's labels.
- **Human–human reliability precondition.** Cohen's kappa is computed between the two human annotators before any comparison with the LLM judge. If human–human kappa falls below 0.6 (the conventional threshold for "substantial agreement" on the Landis–Koch scale), the rubric is judged too unreliable to serve as a reference standard; it is revised and re-piloted on a fresh calibration set before Study 2 proceeds to the judge comparison.
- **Adjudication.** Where the two annotators disagree, an adjudicated label is produced through discussion. This adjudicated label, not either individual annotator's label, is the human reference standard used against the LLM judge.

The LLM judge labels the same 30 items using the same rubric text, without access to either annotator's labels, in a single pass.

5) Statistical Analysis Plan: Two kappa statistics are reported as point estimates over the 30 labeled items, without a bootstrap confidence interval: at $n = 30$, a 2,000-resample bootstrap was judged unlikely to add reliable information

relative to the roadmap's simplified reporting plan, so it is dropped here rather than reported as if it had been run.

- *Human–human kappa*, comparing the two annotators before adjudication, reported as evidence that the rubric is applied reliably;
- *Judge–human kappa*, comparing the LLM judge's labels with the adjudicated human labels, reported as the paper's central validity result.

Both statistics are reported alongside the full confusion matrix and class counts, since kappa can be misleading when one category dominates the sample, and raw observed percent agreement is reported alongside kappa for the same reason.

V. Results

A. Study 1 Results

On the full 97-task set, ROUGE-L F1 has mean 0.2513 and median 0.0: 31 of the 97 responses have nonzero lexical overlap with their reference answer (ranging from 0.043 to 1.0, with 20 of those 31 scoring a perfect 1.0), while the remaining 66 score exactly 0. This is consistent with a single-call agent backed only by DuckDuckGo search snippets failing GAIA's multi-hop lookups outright on roughly two-thirds of tasks, rather than producing near-miss phrasing that ROUGE-L would underscore – i.e. in this sample ROUGE-L's known weakness (penalizing correct paraphrase) is not the dominant failure mode; the agent itself is, though the larger 97-task sample also shows the agent succeeding outright (F1 $= 1.0$) far more often than the earlier 18-task pilot suggested.

On the 8-task, $k = 3$ SelfCheckGPT subset, mean consistency is 0.4808 (individual scores: 0.0 (two tasks), 0.077, 0.333, 0.436, and 1.0 (three tasks)). Per-task Bradley–Terry fitting, after the position-bias control described in Section IV-C, produced at least one usable comparison for 5 of the 8 tasks; the remaining 3 tasks yielded zero position-consistent judge verdicts across all three sample pairs, so no ranking could be fit for them. Comparing the 8 SelfCheckGPT consistency scores against the corresponding Study 2 human hallucination labels (Section V-B) shows no simple correspondence, since near-identical wrong numeric guesses across resamples can be scored as consistent by the lexical/semantic overlap check even though they are all incorrect. At $n = 8$ this is reported descriptively as a limitation of the consistency signal in this sample, not as a formal negative correlation result.

Backend comparison. On the stratified 50-task subset run through both backends, Gemini's ROUGE-L F1 has mean 0.2241 (median 0.0, 16 of 50 nonzero) versus Groq's mean 0.2444 on the identical 50 tasks (median 0.0, 17 of 50 nonzero). The two backends' scores are close and both distributions are dominated by exact-zero failures on roughly two-thirds of tasks, so this small, non-bootstrapped difference is reported descriptively rather than as evidence of a reliable backend effect; it is consistent with Section IV-C's conclusion that the dominant failure mode is the single-call, snippet-only agent architecture rather than a weakness specific to either backend LLM.

B. Study 2 Results

Human–human Cohen's kappa on the 30-item sample is $\kappa = 0.7047$ (substantial agreement on the Landis–Koch scale), with observed agreement 0.8333. This is above the paper's pre-registered 0.6 threshold, so the rubric was not revised or re-piloted. The two annotators disagreed on 5 of 30 items, all instances of *incorrect* vs. *hallucinated*; these were adjudicated by discussion to produce the human reference standard.

Judge–human kappa against the adjudicated labels is $\kappa = 0.1327$ (slight agreement), with observed agreement 0.4333. Table I gives the confusion matrix. The judge agrees with the human standard on every *correct* item (2/2) and every *incorrect* item (11/11), but never outputs *hallucinated*: all 17 items the human standard labeled *hallucinated* were labeled *incorrect* by the judge instead, and *partial* was not used by either side in this sample. The low kappa is therefore driven by one specific, legible failure rather than by generally poor judge behavior: the judge reliably distinguishes right from wrong answers but does not distinguish a specific unsupported/fabricated claim from a plainly wrong one, collapsing two of the rubric's four categories into one.

TABLE I
JUDGE–HUMAN CONFUSION MATRIX ($n = 30$; ROWS = ADJUDICATED HUMAN LABEL, COLUMNS = JUDGE LABEL)

	correct	incorrect	hallucinated
correct	2	0	0
incorrect	0	11	0
hallucinated	0	17	0

VI. Threats to Validity

Threats specific to Study 1 include the small task set for two of its three analyses – only 8 tasks for the SelfCheckGPT/Bradley–Terry subset and 50 tasks for the cross-backend comparison, against 97 for the main ROUGE-L run – relative to the diversity of real agent deployments, and the fact that a single agent configuration is evaluated as the primary analysis rather than two, so Study 1 mostly reports how the four methods behave on one agent, with the Gemini comparison serving only as a secondary, non-bootstrapped check rather than a full comparative ranking across designs. Near-zero ROUGE-L scores on 66 of 97 tasks mean the reference-based metric has little room left to discriminate among responses on a majority of this sample, which limits what can be concluded here about ROUGE-L's sensitivity specifically, as distinct from the agent's accuracy. Threats specific to Study 2 include the small number of human annotators (two, who are also the authors), mitigated by the calibration round, blind independent labeling, the human–human reliability precondition, and adjudication described above, though this cannot fully remove shared bias; prompt sensitivity of the LLM judge, mitigated by fixing the rubric text and prompt across all 30 items; and the small sample size ($n = 30$), reported as point kappa estimates without a bootstrap confidence interval rather than an inflated 2,000-resample interval that the sample size does not support –

this is the paper's main statistical limitation and should be read as such. Study 2's annotated pool is drawn from the earlier, harder-performing 18-task pilot (where only 2 of 18 responses had nonzero ROUGE-L) rather than the full 97-task corpus (where 31 of 97 are nonzero), so the annotated sample may skew toward the agent's failure cases relative to the fuller Study 1 distribution. The SelfCheckGPT re-samples included in Study 2's annotation pool come from tasks outside the original 18-task pilot, so Study 2's pool mixes two corpus vintages. Both studies draw tasks from a single benchmark family (GAIA); Study 2 and Study 1's core (ROUGE-L/SelfCheckGPT/Bradley–Terry) analyses use a single LLM backend (Groq-hosted Llama 3.3 70B) for both the agent and the judge, and only the supplementary 50-task backend comparison introduces a second backend (Gemini, free tier), so results are reported as a case study of the four methods on one benchmark and predominantly one model family rather than a claim of generality across agent types, task domains, or judge models.

ACKNOWLEDGMENT

We thank our Sir, Rajad Shakya, for his help and guidance.

REFERENCES

[1] C.-Y. Lin, "ROUGE: A package for automatic evaluation of summaries," in *Proc. ACL Workshop on Text Summarization Branches Out*, Barcelona, Spain, 2004, pp. 74–81.

[2] R. A. Bradley and M. E. Terry, "Rank analysis of incomplete block designs: I. The method of paired comparisons," *Biometrika*, vol. 39, nos. 3–4, pp. 324–345, 1952.

[3] A. E. Elo, *The Rating of Chessplayers, Past and Present*. New York, NY, USA: Arco, 1978.

[4] L. Zheng *et al.*, "Judging LLM-as-a-judge with MT-Bench and Chatbot Arena," arXiv:2306.05685, 2023.

[5] P. Manakul, A. Liusie, and M. J. F. Gales, "SelfCheckGPT: Zero-resource black-box hallucination detection for generative large language models," in *Proc. EMNLP*, 2023, pp. 9004–9022.

[6] D. Li *et al.*, "From generation to judgment: Opportunities and challenges of LLM-as-a-judge," arXiv preprint, 2024.

[7] H. Li *et al.*, "LLMs-as-judges: A comprehensive survey on LLM-based evaluation methods," arXiv:2412.05579, 2024.

[8] J. Gu *et al.*, "A survey on LLM-as-a-judge," arXiv:2411.15594, 2025.

[9] J. Cohen, "A coefficient of agreement for nominal scales," *Educational and Psychological Measurement*, vol. 20, no. 1, pp. 37–46, 1960.

[10] S. Tan, S. Zhuang, K. Montgomery, W. Y. Tang, A. Cuadron, C. Wang, R. A. Popa, and I. Stoica, "JudgeBench: A benchmark for evaluating LLM-based judges," in *Proc. Int. Conf. on Learning Representations (ICLR)*, 2025.

[11] M. Zhuge *et al.*, "Agent-as-a-Judge: Evaluate agents with agents," arXiv:2410.10934, 2024.

[12] X. H. Lu, A. Kazemnejad, N. Meade, A. Patel, D. Shin, A. Zambrano, K. Stańczak, P. Shaw, C. J. Pal, and S. Reddy, "AgentRewardBench: Evaluating automatic evaluations of web agent trajectories," arXiv:2504.08942, 2025.

[13] G. Mialon, C. Fourrier, C. Swift, T. Wolf, Y. LeCun, and T. Scialom, "GAIA: A benchmark for general AI assistants," arXiv:2311.12983, 2023.